Prompt Saturation: Quality Plateaus at Task-Specific Token Thresholds in Large Language Models

Anonymous authors
Paper under double-blind review

Abstract

Practitioners routinely craft increasingly verbose prompts under the assumption that more context yields better responses. We challenge this assumption with a controlled *saturation experiment*: for six diverse tasks and seven large language models, we measure response quality at seven additive prompt-length levels and fit quality-versus-token curves to identify *saturation points*, the token counts at which quality reaches 95% of its asymptote. Across 5,875 LLM-judge-evaluated responses, we find a consistent **structured–open dichotomy**: tasks requiring structured output (classification, product extraction) exhibit clear saturation curves (F-test $p < 0.05$ in 7 of the $2 \times 7 = 14$ structured model–task pairs; mean $R^2 = 0.85$ for significant fits), with classification saturating at just 38–64 tokens. Product extraction saturates over a wider range (92–536 tokens), with stronger models requiring far fewer tokens, illustrating that saturation point is jointly determined by task complexity and model capability. In contrast, open-ended or reasoning-heavy tasks (question answering, math reasoning) show no significant curve improvement over a constant-quality baseline (0 of 14 open-ended pairs), indicating that quality is bottlenecked by task difficulty rather than prompt elaboration. Model capability modulates saturation point within structured tasks: stronger models require fewer tokens to reach asymptote. These findings provide actionable guidance: structured-output prompts can be aggressively trimmed after the saturation threshold, while extra context in open-ended prompts yields negligible return.

1 Introduction

The rapid adoption of large language models (LLMs) in production systems has generated a body of prompt engineering heuristics: add role descriptions, include worked examples, append step-by-step instructions, and specify output formats in detail (Brown et al., 2020; Wei et al., 2022; Zhou et al., 2023). These practices implicitly assume a monotone relationship between prompt elaboration and response quality. But is this assumption empirically justified, and if so, *up to what point*?

Token costs are not merely economic. Longer prompts consume more inference compute, fill context windows faster, and can *degrade* performance when key information is buried in irrelevant context (Liu et al., 2024). The prompt compression literature (Jiang et al., 2023; Pan et al., 2024) has demonstrated that many tokens are expendable without quality loss, but these methods operate on already-verbose prompts and assume compression is safe. A more fundamental question is: *when does prompt elaboration stop helping in the first place?*

We address this through a **saturation experiment**. For each of six tasks we construct seven strictly additive prompt templates: each template is a proper superset of the previous, adding exactly one layer of elaboration (task label, output format, field definitions, persona, worked example, etc.). This design holds *what* is communicated constant and varies only *how much*. We run every (model, task, level, example) combination for seven frontier LLMs and fit quality-versus-token curves to measure where quality plateaus.

Contributions.

1. A reusable **additive prompt-level methodology** for measuring quality saturation across tasks and models, with open-source code and data.
2. Empirical evidence of a **structured–open dichotomy**: structured-output tasks (classification, product extraction) show statistically significant saturation; open-ended and reasoning-heavy tasks (QA, math reasoning) do not.
3. **Saturation point estimates with bootstrap confidence intervals** for 7 models $\times$ 6 tasks, providing task-specific token budgets for practitioners.
4. A **mechanistic hypothesis**: structured tasks saturate because schema compliance is a discrete capability: once triggered by sufficient elaboration, additional instruction tokens contribute no further signal.

2 Related work

Prompt compression. LLMLingua (Jiang et al., 2023) and LLMLingua-2 (Pan et al., 2024) show that prompts can be compressed 3–20$\times$ with minimal quality loss, demonstrating substantial token redundancy. Chevalier et al. (2023) train models to summarise prompts into soft tokens. More recent pipelines such as CompactPrompt (Wang et al., 2025) combine token pruning with n-gram abbreviation for end-to-end compression. These methods *assume* some prompt content is dispensable; our work provides the first controlled characterisation of *how much* is dispensable as a function of task type.

Prompt engineering. Chain-of-thought (Wei et al., 2022) and few-shot (Brown et al., 2020) prompting add tokens in exchange for quality gains on reasoning tasks. Zhou et al. (2023) show that task-description quality matters more than length for many tasks. Recent surveys of prompting strategies (Sahoo et al., 2024; Schulhoff et al., 2024) and automatic prompt optimisation (Amatriain et al., 2025) catalogue dozens of techniques but do not measure their marginal effect as a function of token count. Our work complements these by quantifying the return of each successive elaboration layer.

Long-context performance. Liu et al. (2024) demonstrate that LLMs underutilise information positioned in the middle of long contexts, a degradation effect distinct from our saturation finding, which concerns the marginal benefit of additional *instruction* tokens before any degradation. Levy et al. (2024) and Hsieh et al. (2024) study how performance varies with context length on retrieval and reasoning benchmarks, finding similar non-monotone effects at much larger scales.

Prompt sensitivity and robustness. Webson & Pavlick (2022) and Mizrahi et al. (2024) show that seemingly small prompt changes cause large quality swings. Our design controls for this by holding content constant and varying only elaboration depth, isolating the token-length signal from phrasing effects.

Scaling laws. Neural scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022) describe how model quality grows with compute and training data. We study an analogous phenomenon at the *prompt level*: how quality grows with instruction tokens, and where this growth saturates. To our knowledge, no prior work has fitted and statistically tested saturation curves at this level of granularity across multiple tasks and models.

3 Methodology

3.1 Additive prompt levels

For each task we define seven prompt templates $T_1 \subset T_2 \subset \cdots \subset T_7$, where $\subset$ denotes strict token superset. Each level appends one layer of elaboration to all previous content; no layer modifies earlier text. Table 1 shows the layer structure for all six tasks.

Table 1: Additive prompt layers by task. Each row adds content to all previous rows.

Lv	Layer added	Classification	Prod. Extraction	QA	Math Reasoning	Summarization	Instr. Following
1	Bare input	Raw text only	Product text	Question only	Problem statement	Article text	Topic prompt
2	Task label	"Classify sentiment"	List field names	"Answer the question"	"Solve the problem"	"Summarize this"	Task + constraints
3	Format spec	"Pos/Neg/Neu"	JSON format	"Answer concisely"	"Give a number"	Length target	Format specification
4	Definitions	Label definitions	Field definitions	Domain context	Show-work instr.	Content guidance	Constraint definitions
5	Persona	Expert analyst	Expert extractor	Domain expert	Math tutor	Journalist	Writing expert
6	Guidelines	Mixed/ironic rules	Edge cases	Uncertainty	Step-by-step rules	Quality rules	Formatting rules
7	Example	Worked example	Worked example	Worked example	Worked example	Worked example	Worked example

Table 2: Models evaluated. Parameter counts are listed where publicly disclosed; "undiscl." indicates the provider has not released this information.

Model (alias)	Provider	API	Parameters
llama-3.1-8b-instant	Meta	Groq	8B
qwen/qwen3-32b	Alibaba	Groq	32B
llama-3.3-70b-versatile	Meta	Groq	70B
gemini-2.0-flash	Google	Gemini	undiscl.
moonshotai/kimi-k2	Moonshot	Groq	undiscl.
gpt-4o-mini	OpenAI	OpenAI	undiscl.
claude-haiku-4-5	Anthropic	Anthropic	undiscl.

The additive design is critical: because level $k+1$ contains all tokens of level k plus exactly one new layer, any quality difference between adjacent levels is attributable solely to that layer. Importantly, no layer is filler; every added layer is meaningful instruction that could plausibly improve performance.

3.2 Tasks

We study six tasks spanning a structured-to-open spectrum:

- **Classification**: sentiment classification (positive / negative / neutral) over 20 product reviews and social media posts. Heuristic evaluator: correct label present in response.
- **Product extraction**: extract name, price, brand, category from product listings in JSON format. 20 examples with varying difficulty (buried prices, implicit brand names). Heuristic: fraction of four fields correctly matched.
- **Question answering (QA)**: factoid questions across geography, science, and history. 20 examples with ground-truth strings. Heuristic: word-overlap recall (ROUGE-1).
- **Math reasoning**: 2–3 step arithmetic, rate, and geometry problems. 20 examples (10 easy, 10 medium difficulty). Heuristic: exact numerical match on the last number in the response.
- **Summarization**: abstractive summarisation of news passages. 20 examples with reference summaries. Heuristic: length score + ROUGE-1 recall.
- **Instruction following**: responses must satisfy explicit structural constraints (bullet points, numbered list, single word). 20 examples, each with 1–2 constraints. Heuristic: fraction of constraints satisfied.

Tasks are grouped *post hoc* into **structured** (classification, product extraction) and **open-ended** (QA, math, summarization, instruction following) based on the analysis in Section 4.

3.3 Models

We evaluate seven LLMs across four providers, covering a broad capability and size range (Table 2). All models are accessed via production APIs with default temperature settings. Each response is capped at 512 output tokens.

3.4 Evaluation

LLM judge. Following Zheng et al. (2023), who show that strong LLM judges match human preferences at over 80% agreement, all 5,875 responses are scored by `gpt-4o-mini` on four dimensions (correctness, completeness, reasoning, conciseness), each rated 1–5. The aggregate quality score is the normalised sum: $q = \frac{\sum_i s_i}{20} \in [0, 1]$. We provide task-specific rubrics to the judge (e.g., for classification: focus on whether the label matches; for product extraction: whether all four fields are present and correct).

Heuristic validation. We additionally validate the LLM judge against a task-specific heuristic evaluator (described in Section 3.2) on the full dataset and find Pearson $r = 0.986$, confirming that judge scores are reliable and not a source of noise in the results.

Aggregation. For each (model, task, level) triplet, we aggregate quality across all 20 examples by taking the mean. This yields 7 data points per (model, task) curve.

3.5 Curve fitting

For each (model, task) pair we fit two functional forms to the 7 level means:

$$q_{\log}(x) = a \log(bx) + c, \qquad q_{\text{sig}}(x) = \frac{L}{1 + e^{-k(x - x_0)}} + c \tag{1}$$

where x is the mean prompt token count at each level. We select the form with lower RMSE. The **saturation point** T^* is defined as the smallest x such that:

$$q(x) \geq 0.95 \times \max_x q(x) \tag{2}$$

Null model F-test. To test whether a fitted curve is significantly better than a flat line ($q(x) = \bar{q}$), we compute:

$$F = \frac{(SS_{\text{null}} - SS_{\text{fit}})/(k-1)}{SS_{\text{fit}}/(n-k)} \tag{3}$$

where $k \in \{3, 4\}$ is the number of curve parameters (log or sigmoid), $n = 7$ is the number of level means, and we use $\alpha = 0.05$. The null model is a constant $q(x) = \bar{q}$, , prompt length has no effect on quality; a significant F-test rejects this in favour of the fitted curve.

Bootstrap confidence intervals. We estimate 95% CIs on T^* via 1,000-iteration bootstrap. On each iteration we resample the 20 example indices *with replacement* and apply the same resample across all seven levels, preserving the paired structure. We re-aggregate, re-fit, and re-compute T^* for each bootstrap replicate; the 2.5th and 97.5th percentiles of the resulting distribution form the CI.

3.6 Experimental scale

Total API calls: $7 \times 6 \times 7 \times 20 = 5{,}880$, with 5,875 successful records (99.9%). All prompts, code, and data are included in the supplementary material.

4 Results

4.1 Structured tasks show clear saturation

Figure 1 shows quality-versus-token curves for all six tasks. Classification curves rise steeply at low token counts and flatten well before level 7, with four of seven models producing F-test-significant saturating curves ($p < 0.05$; Table 3). Saturation points for these four models range from 42 to 64 tokens, well below a typical worked-example prompt (level 7: $\approx$ 300–400 tokens).

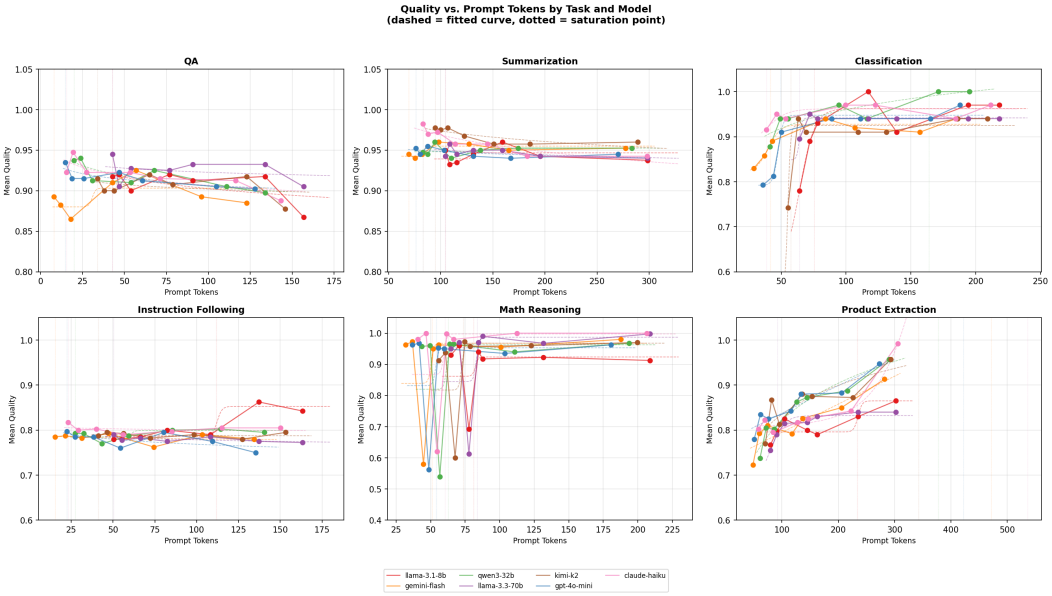

Figure 1: Quality (LLM judge, normalised to [0,1]) vs. mean prompt tokens across seven levels, for all six tasks and seven models. Dashed lines show the best-fit log/sigmoid curve; dotted verticals mark the 95%-of-asymptote saturation point. Classification and product extraction show clear sigmoid curves; QA and math reasoning are flat.

Table 3: F-test results and saturation points. Bold rows are significant ($p < 0.05$). CI: 95% bootstrap confidence interval on saturation token count T^*.

Task	Model	F	p	T^*	95% CI
Classification	**gemini-flash**	14.5	0.027	**42**	[29, 147]
	kimi-k2	21.1	0.016	**57**	[55, 237]
	gpt-4o-mini	42.0	0.006	**50**	[46, 230]
	llama-3.3-70b	22.3	0.015	**64**	[63, 72]
Product Extraction	**llama-3.3-70b**	16.4	0.023	**92**	[83, 312]
	qwen3-32b	26.6	0.005	**378**	[118, 506]
	claude-haiku	49.6	0.005	**536**	[233, 586]
Summarization	**claude-haiku**	9.4	0.031	**83**	[78, 87]
Instr. Following	**llama-3.1-8b**	13.1	0.031	**112**	[50, 314]
QA (all 7)	All $p > 0.12$; not significant				—
Math (all 7)	All $p > 0.70$; not significant				—

Product extraction curves are similarly sigmoidal: three of seven models are significant, with saturation points ranging from 92 (llama-3.3-70b) to 536 (claude-haiku) tokens. The wider range reflects greater schema complexity: a four-field JSON extraction requires more elaboration cues than binary sentiment labelling. For all significant product extraction pairs, $R^2 > 0.70$, confirming strong curve fit.

4.2 Open-ended tasks show no significant saturation

QA and math reasoning yield zero significant pairs out of seven models each. Fitted curves have low R^2 (QA: 0.06–0.69; math: 0.12–0.34) and fail to beat the flat null model at $\alpha = 0.05$ in every case. The quality-versus-token lines appear near-horizontal in Figure 1.

This is a *meaningful negative result*: the same experimental setup detects strong saturation in classification, ruling out methodological artefact. Two interpretations are consistent with

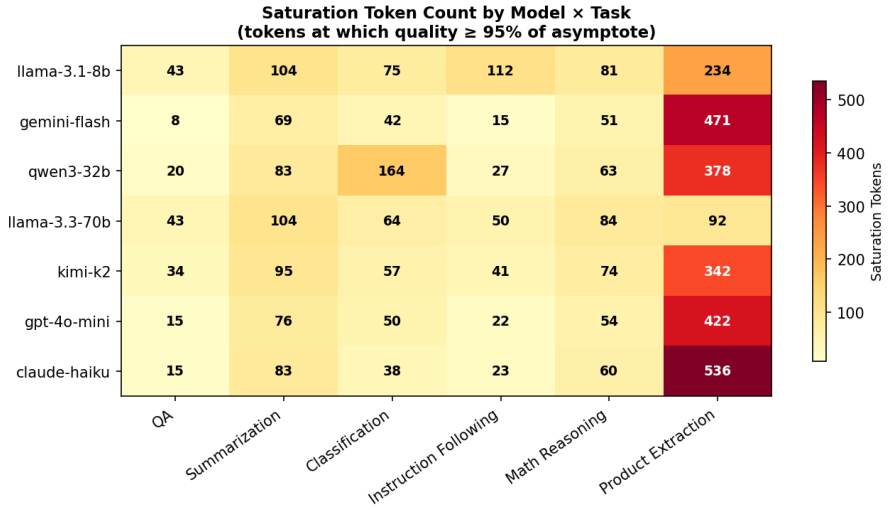

Figure 2: Saturation token counts (heatmap, model × task). Darker shading indicates more tokens needed for saturation. Cells for non-significant fits are shown in grey.

the data: (1) models that already score above 0.85 quality at level 1 (the bare prompt) are near-ceiling, so additional elaboration cannot improve further; (2) models scoring well below ceiling at level 1 are bottlenecked by parametric knowledge or reasoning depth, neither of which is improved by instruction tokens.

Summarization (1/7 significant, claude-haiku only) and instruction following (1/7 significant, llama-3.1-8b only) occupy an intermediate position, consistent with their partial schema requirements.

4.3 Saturation points vary by task and model

Figure 2 shows saturation token counts as a model × task heatmap. Within classification, points range from 42 (gemini-flash) to 64 tokens (llama-3.3-70b) for significant pairs. Within product extraction, the range is substantially wider (92–536 tokens), reflecting greater schema complexity and higher variance across models.

Notably, llama-3.3-70b saturates product extraction at just 92 tokens (CI: [83, 312]), substantially earlier than claude-haiku (536 tokens; CI: [233, 586]). This is consistent with capability differences: a stronger model infers the output schema from fewer cues.

4.4 Forest plot of significant saturation points

Figure 3 plots all nine significant saturation points with 95% bootstrap CIs. Classification CIs are generally narrow (llama-3.3-70b: [63, 72]), indicating precise saturation point estimation. Product extraction CIs are wider, reflecting higher inter-example variance in how models respond to elaboration.

5 Discussion

Why do structured tasks saturate? We hypothesise that structured-output quality is governed primarily by *schema compliance*, the model's ability to produce a response in the required format (e.g., a sentiment label or a four-field JSON object). Schema compliance is a near-discrete capability: the model either produces the correct format or it does not. Once prompt elaboration crosses the threshold that activates this capability, additional instruction tokens provide no further signal. This is consistent with the sigmoid shape of classification

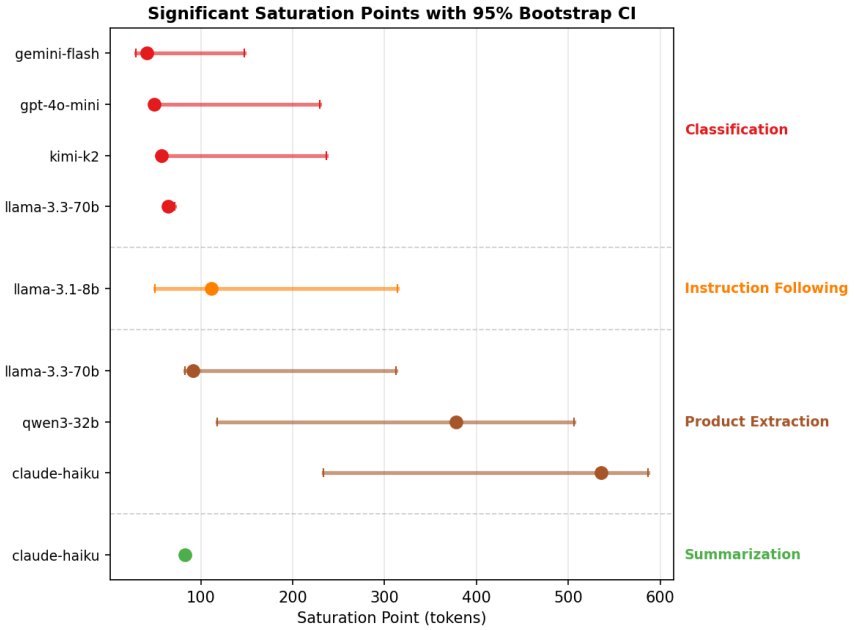

Figure 3: Saturation points with 95% bootstrap CIs (only F-test significant pairs, $p < 0.05$). Classification concentrates below 70 tokens (left); product extraction spans a wider range.

and product extraction curves: a steep rise as the model "activates" the schema, followed by a plateau.

Why do open-ended tasks not saturate? Open-ended tasks require the model to retrieve factual knowledge (QA) or execute multi-step reasoning (math). These capabilities are bounded by what was learned during pre-training, not by how clearly the task is described. Adding more instruction tokens cannot supply knowledge that was never acquired, nor can it increase the model's effective reasoning depth. At level 1 (bare input), capable models already provide near-correct answers; weaker models fail not because they misunderstood the task, but because they lack the requisite knowledge or reasoning. In both cases, further elaboration is uninformative.

Practical guidelines. Our saturation points use a 95%-of-asymptote threshold, making them conservative estimates. Practitioners can use these as upper bounds for prompt budget allocation:

- **Sentiment classification**: a concise task description with label definitions (≈50 tokens) is sufficient for most frontier models; adding a worked example (≈300+ tokens) provides no benefit.
- **Structured extraction**: models vary substantially (92–536 tokens). Weaker or smaller models may require more schema detail; stronger models can infer the schema from minimal cues.
- **QA and math reasoning**: prompt length does not predict quality. Invest instead in retrieval-augmented generation (for QA) or chain-of-thought formatting (for reasoning), which improve quality through mechanisms other than instruction elaboration.

Model capability modulates saturation. Within classification, stronger models (gemini-flash, gpt-4o-mini) saturate at lower token counts (≈42–50 tokens) than the 70B Llama model (64 tokens). The 8B Llama model fails to produce a significant curve at all, suggesting it may require even more elaboration, or that its classification quality is too noisy for a clean

saturation signal. These gradations suggest that saturation points could serve as a *diagnostic* for model instruction-following sensitivity.

5.1 Limitations

Statistical power. With only $n = 7$ data points per curve, the F-test has limited power. Some pairs with genuine saturation may be missed; the low-significance borderline cases (classification for llama-3.1-8b at $p = 0.079$; product extraction for kimi-k2 at $p = 0.093$) are likely true positives that would reach significance with more levels. Future work should use finer-grained level spacing (e.g., 15–20 levels).

Single judge model. All quality scores are produced by `gpt-4o-mini`. Although we validate against heuristic metrics ($r = 0.986$), we do not validate against human raters. Judge bias favouring certain response styles could systematically inflate or deflate quality estimates, though the cross-evaluator agreement suggests this is minimal.

English only. All prompts and examples are in English. Saturation behaviour may differ for low-resource languages where prompt elaboration is more necessary to disambiguate tasks.

Level ordering. Our results reflect a specific ordering of elaboration layers (task label before format before definitions before persona before example). A different ordering might produce different saturation points. Studying the effect of layer permutation is left for future work.

6 Conclusion

We have presented a controlled study of prompt saturation, the point at which response quality stops improving as prompt length increases. Using a seven-level additive prompt methodology across six tasks and seven LLMs, we find a consistent **structured–open dichotomy**: classification saturates at 42–64 tokens, product extraction at 92–536 tokens, while QA and math reasoning show no statistically significant saturation at all.

These findings support a nuanced view of prompt engineering: more tokens help *up to a task-specific threshold* for tasks requiring schema compliance, and *not at all* for tasks bottlenecked by model knowledge or reasoning. The practical implication is direct: prompt budgets should be set by task type, not by general verbosity heuristics.

Future directions. Extending the study to additional structured tasks (code generation with type annotations, data-to-text generation) would test whether the dichotomy generalises. Mechanistic interpretability analyses of how models process elaboration levels could directly test the schema-compliance hypothesis. Finally, studying saturation in multi-turn settings, where context accumulates across turns, would address the increasingly common deployment pattern of long-horizon LLM agents.

Acknowledgments

Ethics Statement

This work involves only publicly available API services. No personally identifiable information was collected. Benchmark examples are drawn from open datasets.

Reproducibility Statement

All prompts, evaluation code, and data are included in the supplementary material. We disclose the use of LLMs in this work: (1) `gpt-4o-mini` served as the quality judge for all 5,875

responses (Section 3.4); (2) an LLM-based coding assistant was used during implementation of the benchmark infrastructure and analysis scripts. (3) LLM assistance was used during the paper writing process. No LLM was used to originate research ideas.

References

Xavier Amatriain et al. A survey of automatic prompt engineering: An optimization perspective. *arXiv preprint arXiv:2502.11560*, 2025.

Tom Brown et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 2020.

Alexis Chevalier et al. Adapting language models to compress contexts. *Proceedings of EMNLP*, 2023.

Jordan Hoffmann et al. Training compute-optimal large language models. *Advances in Neural Information Processing Systems*, 35, 2022.

Cheng-Ping Hsieh et al. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

Huiqiang Jiang et al. LLMLingua: Compressing prompts for accelerated inference of large language models. *Proceedings of EMNLP*, 2023.

Jared Kaplan et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Mosh Levy et al. Same task, more tokens: The impact of input length on the reasoning performance of large language models. *arXiv preprint arXiv:2402.14848*, 2024.

Nelson F. Liu et al. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

Moran Mizrahi et al. State of what art? A call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12, 2024.

Zhuoshi Pan et al. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *arXiv preprint arXiv:2403.12968*, 2024.

Pranab Sahoo et al. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*, 2024.

Sander Schulhoff et al. The prompt report: A systematic survey of prompting techniques. *arXiv preprint arXiv:2406.06608*, 2024.

Zhuo Wang et al. CompactPrompt: A unified pipeline for prompt and data compression in LLM workflows. *arXiv preprint arXiv:2510.18043*, 2025.

Albert Webson and Ellie Pavlick. Do prompt-based models really understand the meaning of their prompts? *Proceedings of NAACL*, 2022.

Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.

Lianmin Zheng et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Yongchao Zhou et al. Large language models are human-level prompt engineers. *International Conference on Learning Representations*, 2023.

A Full prompt level specifications

A.1 Classification

1. Bare text input only.
2. Add task label: "Classify the sentiment of the following text as Positive, Negative, or Neutral."
3. Add format constraint: "Respond with only the single label word."
4. Add label definitions: Positive = expresses approval or satisfaction; Negative = expresses dissatisfaction or criticism; Neutral = neither or mixed.
5. Add persona: "You are an expert sentiment analyst."
6. Add guidelines: rules for ironic text, mixed sentiment, and domain jargon.
7. Add worked example: a complete input/output example with explanation.

A.2 Product extraction

1. Bare product listing text only.
2. List required output fields: name, price, brand, category.
3. Specify JSON output format with exact key names.
4. Define each field: name = product title, price = numeric value without currency symbol, brand = manufacturer name, category = product type.
5. Add persona (expert product data analyst) and missing-field instruction (use null for absent fields).
6. Add extraction guidelines for ambiguous cases: implied brand names, price ranges, composite product names.
7. Add full worked example: verbose product description with correct JSON output.

A.3 Question answering

1. Question only.
2. Add task label: "Answer the following question."
3. Add format guidance: "Be concise and direct."
4. Add domain context appropriate to the question.
5. Add expert persona.
6. Add uncertainty instruction: express uncertainty explicitly if unsure.
7. Add worked example.

A.4 Math reasoning

1. Problem statement only.
2. Add task label: "Solve the following math problem."
3. Add answer format: "Give a single numeric answer."
4. Add show-work instruction: "Show your reasoning step by step."
5. Add math tutor persona.
6. Add step-by-step guidelines and common error warnings.
7. Add worked example with full solution.

A.5 Summarization

1. Article text only.
2. Add task label: "Summarize the following article."
3. Add length target: "Write a summary of approximately 50–80 words."
4. Add content guidance: include who, what, when, where.
5. Add journalist persona.
6. Add quality guidelines: avoid redundancy, preserve key facts.
7. Add worked example: article with reference summary.

A.6 Instruction following

1. Topic prompt only ("List benefits of exercise").
2. Add task label and constraints explicitly stated.
3. Add format specification mirroring the constraint type.
4. Add constraint definitions (e.g., bullet = "-" or "*" prefix).
5. Add writing expert persona.
6. Add formatting guidelines and error examples.
7. Add worked example satisfying all constraints.

B LLM judge rubric

All 5,875 responses are scored by `gpt-4o-mini` using the following prompt template. The {rubric} placeholder is filled with a task-specific rubric (Table 4).

```
You are an expert judge evaluating an AI assistant's response.

Task type: {task_type}
Evaluation guidance: {rubric}
Reference answer (if available): {ground_truth}

AI response to evaluate:
{response}

Rate the response on these 4 dimensions, each from 1 (very poor) to 5
(excellent):
- correctness: Does the response accurately answer the task?
- completeness: Is the response complete and thorough?
- reasoning: Is the reasoning or logic sound and clear?
- conciseness: Is the response appropriately concise?

Respond ONLY with valid JSON in this exact format:
{"correctness": <1-5>, "completeness": <1-5>, "reasoning": <1-5>,
"conciseness": <1-5>}
```

C Classification prompt example (all seven levels)

Below we show the full prompt text at each of the seven additive levels for a single classification input: *"The product works great and I'm very happy with my purchase."* Each level is a strict superset of the previous.

Level 1 (bare input).

```
Classify: The product works great and I'm very happy with my purchase.
```

Level 2 (+ class names).

```
Classify sentiment as positive, negative, or neutral: The product works
great and I'm very happy with my purchase.
```

Table 4: Task-specific rubrics provided to the LLM judge.

Task	Rubric
Classification	Focus on Correctness (does the classification match the reference label?). Conciseness rewards direct labelling without padding. Reasoning Quality assesses any justification given. Completeness checks if the classification is unambiguous.
Product Extraction	Focus on Correctness (do the extracted fields match the reference values?). Completeness checks if all 4 fields (name, price, brand, category) are present. Reasoning Quality is less relevant; focus on extraction accuracy. Conciseness rewards clean JSON output without extra text.
QA	Focus heavily on Correctness (does the answer match the reference?). Completeness checks if the answer is fully given. Reasoning Quality assesses logical justification. Conciseness penalises unnecessary verbosity.
Math Reasoning	Focus on Correctness (does the final numerical answer match the reference?). Completeness checks if work is shown. Reasoning Quality assesses whether the solution steps are logical. Conciseness penalises unnecessary verbosity beyond the solution.
Summarization	Focus on Completeness (coverage of key points from the reference). Correctness checks factual accuracy. Conciseness rewards brevity over length. Reasoning Quality assesses coherence and structure.
Instr. Following	Focus on Completeness (are all stated constraints satisfied?). Correctness checks format and structural adherence. Conciseness rewards appropriate brevity. Reasoning Quality assesses overall response clarity.

Level 3 (+ output format).

```
Classify sentiment as positive, negative, or neutral. Respond with only
the label: The product works great and I'm very happy with my purchase.
```

Level 4 (+ label definitions).

```
Classify the sentiment of the following text as positive, negative, or
neutral. Respond with only the label.
Definitions:
- positive: overall favorable or optimistic tone
- negative: overall unfavorable or critical tone
- neutral: balanced, factual, or no clear sentiment
Text: The product works great and I'm very happy with my purchase.
```

Level 5 (+ edge-case handling).

```
Classify the sentiment of the following text as positive, negative, or
neutral. Respond with only the label.
Definitions:
- positive: overall favorable or optimistic tone
- negative: overall unfavorable or critical tone
- neutral: balanced, factual, or no clear sentiment
Edge cases: If the text contains mixed sentiment, choose the dominant tone.
If equally mixed, use neutral.
Text: The product works great and I'm very happy with my purchase.
```

Level 6 (+ persona and full guidelines).

```
You are a sentiment classification expert. Classify the sentiment of the
following text as positive, negative, or neutral.
Rules:
```

```
1. Respond with ONLY one word: positive, negative, or neutral
2. Base your judgment on the overall tone, not individual words
3. Positive: clearly favorable, optimistic, praising, or satisfied
4. Negative: clearly unfavorable, critical, pessimistic, or dissatisfied
5. Neutral: factual reporting, balanced views, or no discernible sentiment
6. If mixed, choose the dominant sentiment; if equal, use neutral
7. Take text at face value; do not attempt sarcasm detection

Text: The product works great and I'm very happy with my purchase.
```

Level 7 (+ worked example).

```
You are a sentiment classification expert. Classify the sentiment of the
following text as positive, negative, or neutral.

Rules:
1. Respond with ONLY one word: positive, negative, or neutral
2. Base your judgment on the overall tone, not individual words
3. Positive: clearly favorable, optimistic, praising, or satisfied
4. Negative: clearly unfavorable, critical, pessimistic, or dissatisfied
5. Neutral: factual reporting, balanced views, or no discernible sentiment
6. If mixed, choose the dominant sentiment; if equal, use neutral
7. Take text at face value; do not attempt sarcasm detection

Example:
Text: The product works great and I'm very happy with my purchase.
Label: positive

Now classify:
Text: The product works great and I'm very happy with my purchase.
```

D Heuristic vs. LLM judge validation

To verify that the LLM judge scores are reliable, we compute the Pearson correlation between the task-specific heuristic quality score (Section 3.2) and the normalised judge score ($q = \sum_i s_i / 20$) across all records where both are available. The overall correlation is $r = 0.986$ ($p < 0.0001$). Table 5 reports per-strategy breakdowns, confirming that agreement holds uniformly across prompting conditions.

Table 5: Pearson correlation between heuristic and LLM judge quality scores, overall and per prompting strategy.

Subset	Pearson r
Overall	0.986
zero_shot	0.985
zero_shot_verbose	0.983
few_shot	0.980
chain_of_thought	0.984
concise	0.987

E Per-model saturation statistics

Table 6 reports fit type, R^2, F-test p-value, and saturation token count for all 42 (model, task) pairs.

Table 6: Complete saturation statistics. Bold: F-test $p < 0.05$. Fit = best-fit curve type (log = logarithmic, sig = sigmoid).

Model	Task	Fit	R^2	p	T^*
llama-3.1-8b	QA	log	0.291	0.503	43
llama-3.1-8b	Summarization	sig	0.161	0.896	104
llama-3.1-8b	Classification	sig	0.866	0.079	75
llama-3.1-8b	**Instr. Following**	sig	**0.929**	**0.031**	**112**
llama-3.1-8b	Math Reasoning	sig	0.125	0.928	81
llama-3.1-8b	Product Extraction	sig	0.720	0.229	234
gemini-flash	QA	sig	0.404	0.622	8
gemini-flash	Summarization	sig	0.760	0.185	69
gemini-flash	**Classification**	sig	**0.936**	**0.027**	**42**
gemini-flash	Instr. Following	log	0.067	0.871	15
gemini-flash	Math Reasoning	sig	0.205	0.853	51
gemini-flash	Product Extraction	sig	0.830	0.113	471
qwen3-32b	QA	log	0.690	0.096	20
qwen3-32b	Summarization	sig	0.106	0.944	83
qwen3-32b	Classification	log	0.753	0.061	164
qwen3-32b	Instr. Following	sig	0.465	0.544	27
qwen3-32b	Math Reasoning	sig	0.217	0.840	63
qwen3-32b	**Product Extraction**	log	**0.930**	**0.005**	**378**
llama-3.3-70b	QA	log	0.065	0.875	43
llama-3.3-70b	Summarization	log	0.290	0.503	104
llama-3.3-70b	**Classification**	sig	**0.957**	**0.015**	**64**
llama-3.3-70b	Instr. Following	log	0.379	0.385	50
llama-3.3-70b	Math Reasoning	sig	0.282	0.769	84
llama-3.3-70b	**Product Extraction**	sig	**0.943**	**0.023**	**92**
kimi-k2	QA	log	0.097	0.816	34
kimi-k2	Summarization	log	0.664	0.113	95
kimi-k2	**Classification**	sig	**0.955**	**0.016**	**57**
kimi-k2	Instr. Following	log	0.027	0.947	41
kimi-k2	Math Reasoning	sig	0.339	0.702	74
kimi-k2	Product Extraction	log	0.695	0.093	342
gpt-4o-mini	QA	log	0.646	0.125	15
gpt-4o-mini	Summarization	log	0.356	0.414	76
gpt-4o-mini	**Classification**	sig	**0.977**	**0.006**	**50**
gpt-4o-mini	Instr. Following	log	0.386	0.377	22
gpt-4o-mini	Math Reasoning	sig	0.178	0.879	54
gpt-4o-mini	Product Extraction	sig	0.877	0.071	422
claude-haiku	QA	log	0.618	0.146	15
claude-haiku	**Summarization**	log	**0.824**	**0.031**	**83**
claude-haiku	Classification	sig	0.646	0.317	38
claude-haiku	Instr. Following	log	0.071	0.863	23
claude-haiku	Math Reasoning	sig	0.228	0.829	60
claude-haiku	**Product Extraction**	sig	**0.980**	**0.005**	**536**